```
!pip install tensorflow scikit-learn numpy wget

import json
import numpy as np
import re
import pickle
import os
import wget
import zipfile
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (Input, LSTM, Embedding, Dense, Bidirectional,
                                     Concatenate, Attention, TimeDistributed,
                                     Dropout, LayerNormalization)
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from sklearn.model_selection import train_test_split

# ============== CONFIGURATION ==============
MAX_TEXT_LEN = 400
MAX_SUMMARY_LEN = 60
EMBEDDING_DIM = 100
HIDDEN_UNITS = 256
BATCH_SIZE = 32
EPOCHS = 25
VOCAB_LIMIT = 20000
GLOVE_DIR = './glove.6B'
GLOVE_ZIP = 'glove.6B.zip'
GLOVE_URL = 'http://nlp.stanford.edu/data/glove.6B.zip'

# ============== 1. HELPER: DATA GENERATION ==============
def create_dummy_data(filename='legal_dataset.json'):
    if not os.path.exists(filename):
        print(f"⚠ {filename} not found. Generating dummy data for testing...")
        dummy_data = {
            "documents": [
                {
                    "judgment": {
                        "judgment_text": "The appellant herein was convicted under section 302 of the Indian Penal Code for
                        "summary": "Appellant convicted under Section 302 IPC. Case based on circumstantial evidence."
                    }
                } for _ in range(200)
            ]
        }
        with open(filename, 'w') as f:
            json.dump(dummy_data, f)
        print("✓ Dummy dataset created.")

# ============== 2. HELPER: DOWNLOAD GLOVE ==============
def download_glove():
    if not os.path.exists(GLOVE_DIR):
        print("⚡ Downloading GloVe Embeddings (800MB)...")
        os.makedirs(GLOVE_DIR, exist_ok=True)
        if not os.path.exists(GLOVE_ZIP):
            wget.download(GLOVE_URL, GLOVE_ZIP)
        with zipfile.ZipFile(GLOVE_ZIP, 'r') as zip_ref:
            zip_ref.extractall(GLOVE_DIR)
        print("✓ GloVe Ready.")
    else:
        print("✓ GloVe Embeddings already present.")

def load_glove_embeddings(word_index, vocab_size):
    embeddings_index = {}
    f = open(os.path.join(GLOVE_DIR, 'glove.6B.100d.txt'), encoding="utf-8")
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs
    f.close()

    embedding_matrix = np.zeros((vocab_size, EMBEDDING_DIM))
    hits = 0
    for word, i in word_index.items():
        if i < vocab_size:
            embedding_vector = embeddings_index.get(word)
            if embedding_vector is not None:
                embedding_matrix[i] = embedding_vector
                hits += 1
```

```
    print(f"✓ Embeddings Loaded: {hits} words matched.")
    return embedding_matrix

# ============== 3. DATA LOADING ==============
def load_and_process_data(json_path='legal_dataset.json'):
    create_dummy_data(json_path)
    with open(json_path, 'r', encoding='utf-8') as f:
        data = json.load(f)

    texts = []
    summaries = []
    for doc in data['documents']:
        j_text = doc['judgment']['judgment_text']
        s_text = doc['judgment']['summary']
        j_clean = re.sub(r'[^a-zA-Z0-9\s]', '', j_text.lower())
        s_clean = re.sub(r'[^a-zA-Z0-9\s]', '', s_text.lower())
        texts.append(j_clean)
        summaries.append('sostok ' + s_clean + ' eostok')

    return texts, summaries

# ============== 4. TOKENIZATION ==============
def prepare_tokenizers(texts, summaries):
    text_tokenizer = Tokenizer(num_words=VOCAB_LIMIT, oov_token='<UNK>')
    text_tokenizer.fit_on_texts(texts)

    summary_tokenizer = Tokenizer(num_words=VOCAB_LIMIT, oov_token='<UNK>')
    summary_tokenizer.fit_on_texts(summaries)

    X = text_tokenizer.texts_to_sequences(texts)
    y = summary_tokenizer.texts_to_sequences(summaries)

    X = pad_sequences(X, maxlen=MAX_TEXT_LEN, padding='post')
    y = pad_sequences(y, maxlen=MAX_SUMMARY_LEN, padding='post')

    text_vocab = min(len(text_tokenizer.word_index) + 1, VOCAB_LIMIT)
    summ_vocab = min(len(summary_tokenizer.word_index) + 1, VOCAB_LIMIT)

    return X, y, text_tokenizer, summary_tokenizer, text_vocab, summ_vocab

# ============== 5. ADVANCED MODEL (FIXED) ==============
def build_advanced_model(text_vocab, summary_vocab, embedding_matrix):
    print("Building Stacked BiLSTM + Attention Model...")

    # --- ENCODER ---
    encoder_input = Input(shape=(MAX_TEXT_LEN,), name='enc_input')

    # FIX: Set mask_zero=False to prevent Attention Layer crash
    enc_emb_layer = Embedding(text_vocab, EMBEDDING_DIM,
                              weights=[embedding_matrix],
                              trainable=True,
                              mask_zero=False,  # <--- CRITICAL FIX
                              name='glove_embedding')
    enc_emb = enc_emb_layer(encoder_input)

    # Encoder Layer 1
    encoder_l1 = Bidirectional(LSTM(HIDDEN_UNITS, return_sequences=True, dropout=0.4))
    enc_out1 = encoder_l1(enc_emb)
    enc_out1 = LayerNormalization()(enc_out1)

    # Encoder Layer 2
    encoder_l2 = Bidirectional(LSTM(HIDDEN_UNITS, return_sequences=True, return_state=True, dropout=0.4))
    enc_output, fh, fc, bh, bc = encoder_l2(enc_out1)

    state_h = Concatenate()([fh, bh])
    state_c = Concatenate()([fc, bc])

    # --- DECODER ---
    decoder_input = Input(shape=(MAX_SUMMARY_LEN,), name='dec_input')
    dec_emb_layer = Embedding(summary_vocab, EMBEDDING_DIM, trainable=True)
    dec_emb = dec_emb_layer(decoder_input)

    # Decoder LSTM
    decoder_lstm = LSTM(HIDDEN_UNITS*2, return_sequences=True, return_state=True, dropout=0.4)
    dec_output, _, _ = decoder_lstm(dec_emb, initial_state=[state_h, state_c])

    # --- ATTENTION ---
    attn_layer = Attention(name='attention_layer')
    attn_out = attn_layer([dec_output, enc_output])

    decoder_concat = Concatenate(axis=-1)([dec_output, attn_out])
```

```
    # --- OUTPUT ---
    decoder_dense = TimeDistributed(Dense(summary_vocab, activation='softmax'))
    output = decoder_dense(decoder_concat)

    model = Model([encoder_input, decoder_input], output, name='Advanced_Legal_Summarizer')

    opt = tf.keras.optimizers.Adam(learning_rate=0.001)
    model.compile(optimizer=opt, loss='sparse_categorical_crossentropy', metrics=['accuracy'])

    return model

# ============== MAIN PIPELINE ==============
if __name__ == "__main__":
    download_glove()
    texts, summaries = load_and_process_data('legal_dataset.json')
    X, y, text_tok, summ_tok, text_vocab, summ_vocab = prepare_tokenizers(texts, summaries)

    # Load GloVe
    embedding_matrix = load_glove_embeddings(text_tok.word_index, text_vocab)

    # Split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)
    print(f"\nTraining Samples: {len(X_train)} | Test Samples: {len(X_test)}")

    # Build
    model = build_advanced_model(text_vocab, summ_vocab, embedding_matrix)
    model.summary()

    # Prepare Targets
    dec_in_train = y_train[:, :-1]
    dec_target_train = y_train[:, 1:]
    dec_in_test = y_test[:, :-1]
    dec_target_test = y_test[:, 1:]

    dec_in_train = pad_sequences(dec_in_train, maxlen=MAX_SUMMARY_LEN, padding='post')
    dec_target_train = pad_sequences(dec_target_train, maxlen=MAX_SUMMARY_LEN, padding='post')
    dec_in_test = pad_sequences(dec_in_test, maxlen=MAX_SUMMARY_LEN, padding='post')
    dec_target_test = pad_sequences(dec_target_test, maxlen=MAX_SUMMARY_LEN, padding='post')

    # Callbacks
    es = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=5, restore_best_weights=True)
    ck = ModelCheckpoint('legal_summarizer.keras', monitor='val_loss', save_best_only=True)
    lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2, min_lr=0.00001, verbose=1)

    # Train
    print("\n🚀 Starting Advanced Training...")
    history = model.fit(
        [X_train, dec_in_train],
        np.expand_dims(dec_target_train, -1),
        batch_size=BATCH_SIZE,
        epochs=EPOCHS,
        validation_data=([X_test, dec_in_test], np.expand_dims(dec_target_test, -1)),
        callbacks=[es, ck, lr]
    )

    # Save
    with open('tokenizers.pkl', 'wb') as f:
        pickle.dump({'text': text_tok, 'summary': summ_tok}, f)

    print("\n✅ Training Complete.")
    print("✅ Model Saved: legal_summarizer.keras")
    print("✅ Tokenizers Saved: tokenizers.pkl")
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: wget in /usr/local/lib/python3.12/dist-packages (3.2)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorfl
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/loca
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.2.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10.0)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensor
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.18.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorf
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensor
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorf
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboa
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->te
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->kera
✓ GloVe Embeddings already present.
✓ Embeddings Loaded: 362 words matched.

Training Samples: 180 | Test Samples: 20
Building Stacked BiLSTM + Attention Model...
Model: "Advanced_Legal_Summarizer"
```

Layer (type)	Output Shape	Param #	Connected to
enc_input (InputLayer)	(None, 400)	0	-
glove_embedding (Embedding)	(None, 400, 100)	38,500	enc_input[0][0]
bidirectional_11 (Bidirectional)	(None, 400, 512)	731,136	glove_embedding[…
layer_normalizatio… (LayerNormalizatio…	(None, 400, 512)	1,024	bidirectional_11…

```
!pip install rouge-score nltk

import json
import numpy as np
import pickle
import re
import time
import os
import nltk

# Download necessary NLTK data
nltk.download('punkt')
nltk.download('wordnet')

from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.sequence import pad_sequences
from rouge_score import rouge_scorer
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
import warnings
warnings.filterwarnings('ignore')

# ============== CONFIGURATION ==============
MAX_TEXT_LEN = 400
MAX_SUMMARY_LEN = 60
MODEL_FILENAME = 'legal_summarizer.keras'
```

```
TEST_DOCS_LIMIT = 25     # Test on 25 docs as requested

# ============== LOAD RESOURCES ==============
print("Loading resources...")

if not os.path.exists(MODEL_FILENAME):
    print(f"❌ Error: {MODEL_FILENAME} not found.")
    exit()

model = load_model(MODEL_FILENAME, compile=False)
with open('tokenizers.pkl', 'rb') as f:
    tokenizers = pickle.load(f)
    text_tokenizer = tokenizers['text']
    summary_tokenizer = tokenizers['summary']

print("✓ Model loaded.")

# ============== DATA LOADER ==============
def load_test_data(json_path='legal_dataset.json'):
    with open(json_path, 'r', encoding='utf-8') as f:
        data = json.load(f)

    texts = []
    summaries = []

    for doc in data['documents'][:TEST_DOCS_LIMIT]:
        t_text = doc.get('judgment', {}).get('judgment_text', '')
        s_text = doc.get('judgment', {}).get('summary', '')

        text = re.sub(r'[^a-zA-Z0-9\s]', '', t_text.lower())
        summary = re.sub(r'[^a-zA-Z0-9\s]', '', s_text.lower())

        if text and summary:
            texts.append(text)
            summaries.append(summary)
    return texts, summaries

# ============== PREDICTION ENGINE ==============
def predict_summary(text, model, text_tok, summ_tok):
    seq = text_tok.texts_to_sequences([text])
    padded_input = pad_sequences(seq, maxlen=MAX_TEXT_LEN, padding='post')

    decoder_input = np.zeros((1, MAX_SUMMARY_LEN))
    decoder_input[0, 0] = summ_tok.word_index.get('sostok', 1)

    decoded_words = []
    seen_words = set()

    for i in range(1, MAX_SUMMARY_LEN):
        output = model.predict([padded_input, decoder_input], verbose=0)
        top_indices = np.argsort(output[0, i-1, :])[-5:][::-1]

        predicted_id = top_indices[0]
        word = summ_tok.index_word.get(predicted_id, '')

        # Repetition Check
        if word in seen_words and len(word) > 3:
            predicted_id = top_indices[1]

        if predicted_id == 0: break
        word = summ_tok.index_word.get(predicted_id, '')

        if word == 'eostok': break
        if word != 'sostok':
            decoded_words.append(word)
            seen_words.add(word)

        if i < MAX_SUMMARY_LEN:
            decoder_input[0, i] = predicted_id

    return ' '.join(decoded_words)

# ============== COMPREHENSIVE METRICS ==============
def calculate_comprehensive_metrics(predictions, references):
    # 1. ROUGE Scores
    scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)
    r1, r2, rl = [], [], []

    for pred, ref in zip(predictions, references):
        if not pred.strip(): pred = "empty"
        scores = scorer.score(ref, pred)
        r1.append(scores['rouge1'].fmeasure)
        r2.append(scores['rouge2'].fmeasure)
```

```
            r1.append(scores['rouge1'].fmeasure)
            rl.append(scores['rougeL'].fmeasure)

        # 2. BLEU Score
        smoothing = SmoothingFunction().method1
        bleu = []
        for pred, ref in zip(predictions, references):
            ref_split = ref.split() if ref else ["empty"]
            pred_split = pred.split() if pred else ["empty"]
            bleu.append(sentence_bleu([ref_split], pred_split, smoothing_function=smoothing))

        # 3. Precision, Recall, F1, Exact Match
        precisions, recalls, f1s, exacts = [], [], [], []

        for pred, ref in zip(predictions, references):
            pred_tokens = set(pred.split())
            ref_tokens = set(ref.split())

            # Exact Match
            exacts.append(1 if pred.strip() == ref.strip() else 0)

            if len(pred_tokens) == 0:
                precisions.append(0); recalls.append(0); f1s.append(0)
                continue

            common = len(pred_tokens.intersection(ref_tokens))

            p = common / len(pred_tokens)
            r = common / len(ref_tokens) if len(ref_tokens) > 0 else 0
            f1 = 2 * p * r / (p + r) if (p + r) > 0 else 0

            precisions.append(p)
            recalls.append(r)
            f1s.append(f1)

        return {
            "ROUGE-1": np.mean(r1),
            "ROUGE-2": np.mean(r2),
            "ROUGE-L": np.mean(rl),
            "BLEU": np.mean(bleu),
            "Precision": np.mean(precisions),
            "Recall": np.mean(recalls),
            "F1-Score": np.mean(f1s),
            "Exact_Match": np.mean(exacts)
        }

    # ============== MAIN EXECUTION ==============
    if __name__ == "__main__":
        test_texts, test_summaries = load_test_data()
        print(f"Testing on {len(test_texts)} documents...\n")

        predictions = []
        times = []

        for i, text in enumerate(test_texts):
            start = time.time()
            pred = predict_summary(text, model, text_tokenizer, summary_tokenizer)
            times.append(time.time() - start)
            predictions.append(pred)

            if (i+1) % 5 == 0: print(f"  Processed {i+1}...")

        m = calculate_comprehensive_metrics(predictions, test_summaries)
        avg_time = np.mean(times)

        print("\n" + "="*60)
        print(f"{'METRIC':<20} | {'VALUE':<10} | {'DESCRIPTION'}")
        print("="*60)
        print(f"{'ROUGE-1':<20} | {m['ROUGE-1']:.4f}     | Unigram overlap (Keywords)")
        print(f"{'ROUGE-2':<20} | {m['ROUGE-2']:.4f}     | Bigram overlap (Phrasing)")
        print(f"{'ROUGE-L':<20} | {m['ROUGE-L']:.4f}     | Longest Common Subsequence")
        print("-" * 60)
        print(f"{'BLEU Score':<20} | {m['BLEU']:.4f}     | Standard Translation Metric")
        print("-" * 60)
        print(f"{'Precision':<20} | {m['Precision']:.2%}     | Accuracy of generated words")
        print(f"{'Recall':<20} | {m['Recall']:.2%}     | Coverage of reference words")
        print(f"{'F1-Score':<20} | {m['F1-Score']:.2%}     | Balance of P & R")
        print(f"{'Exact Match':<20} | {m['Exact_Match']:.1%}     | Perfect summary matches")
        print("-" * 60)
        print(f"{'Avg Latency':<20} | {avg_time:.2f}s    | Time per document")
        print("="*60)
```

```
Requirement already satisfied: rouge-score in /usr/local/lib/python3.12/dist-packages (0.1.2)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from rouge-score) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rouge-score) (2.0.2)
Requirement already satisfied: six>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from rouge-score) (1.17.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Loading resources...
✓ Model loaded.
Testing on 25 documents...

  Processed 5...
  Processed 10...
  Processed 15...
  Processed 20...
  Processed 25...

============================================================
METRIC               | VALUE      | DESCRIPTION
============================================================
ROUGE-1              | 0.7564     | Unigram overlap (Keywords)
ROUGE-2              | 0.5972     | Bigram overlap (Phrasing)
ROUGE-L              | 0.7120     | Longest Common Subsequence
------------------------------------------------------------
BLEU Score           | 0.5160     | Standard Translation Metric
------------------------------------------------------------
Precision            | 73.50%     | Accuracy of generated words
Recall               | 72.15%     | Coverage of reference words
F1-Score             | 72.76%     | Balance of P & R
Exact Match          | 0.0%     | Perfect summary matches
------------------------------------------------------------
Avg Latency          | 6.10s    | Time per document
```

```
import numpy as np
import pickle
import re
import os
import textwrap
import sys
import time

# Suppress TF warnings
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.sequence import pad_sequences

# ============== CONFIG (MATCHES ADVANCED MODEL) ==============
MAX_TEXT_LEN = 400       # Updated to match Advanced Training
MAX_SUMMARY_LEN = 60     # Updated to match Advanced Training
MODEL_FILENAME = 'legal_summarizer.keras'
TOKENIZER_FILENAME = 'tokenizers.pkl'

# ============== DATA INPUTS ==============
HARDCODED_CASE = """
The appellant herein was convicted by the Trial Court under Section 302 of the Indian Penal Code (IPC) for the murder of his

Upon arrival, the police found the deceased lying in a pool of blood with severe head injuries. The post-mortem report confi

The appellant pleaded not guilty, claiming alibi, stating he was at his brother's house during the incident. However, the pr

The High Court dismissed the appeal, upholding the life imprisonment sentence awarded by the Sessions Court. The appellant h
"""

HARDCODED_QUESTIONS = [
    "Who was convicted in this case?",
    "What section of IPC was applied?",
    "What was the cause of death?",
    "Did the fingerprints match?",
    "What was the final verdict?"
]

# ============== UI HELPERS (NO COLORS) ==============
def clear_screen():
    os.system('cls' if os.name == 'nt' else 'clear')

def print_header():
    print("="*70)
    print("          LEGAL ASSISTANT AI (BiLSTM + ATTENTION)")
    print("="*70)
```

```
def type_effect(text, delay=0.01):
    """Simulates typing effect"""
    for char in text:
        sys.stdout.write(char)
        sys.stdout.flush()
        time.sleep(delay)
    print()

def loading_bar(description):
    print(f"\n{description}")
    sys.stdout.write("[")
    for _ in range(30):
        sys.stdout.write("=")
        sys.stdout.flush()
        time.sleep(0.05)
    sys.stdout.write("] Done.\n")

# ============== MODEL LOADING ==============
def load_resources():
    print("Loading AI Models... Please wait...")

    if not os.path.exists(MODEL_FILENAME) or not os.path.exists(TOKENIZER_FILENAME):
        print("\nCRITICAL ERROR: Model files missing!")
        sys.exit(1)

    try:
        # compile=False is safer for custom architectures
        model = load_model(MODEL_FILENAME, compile=False)
        with open(TOKENIZER_FILENAME, 'rb') as f:
            tokenizers = pickle.load(f)
        return model, tokenizers['text'], tokenizers['summary']
    except Exception as e:
        print(f"\nError: {e}")
        sys.exit(1)

# ============== 1. SMART SUMMARY GENERATOR ==============
def generate_summary(text, model, text_tok, summ_tok):
    # Preprocess (Pad to 400)
    clean_text = re.sub(r'[^a-zA-Z0-9\s]', '', text.lower())
    seq = text_tok.texts_to_sequences([clean_text])
    padded_input = pad_sequences(seq, maxlen=MAX_TEXT_LEN, padding='post')

    # Setup Decoder
    decoder_input = np.zeros((1, MAX_SUMMARY_LEN))
    start_token = summ_tok.word_index.get('sostok', 1)
    decoder_input[0, 0] = start_token

    decoded_words = []
    seen_words = set()  # Anti-repetition set

    # Generation Loop
    for i in range(1, MAX_SUMMARY_LEN):
        output = model.predict([padded_input, decoder_input], verbose=0)

        # Get Top 3 Candidates
        top_indices = np.argsort(output[0, i-1, :])[-3:][::-1]

        predicted_id = top_indices[0]
        word_candidate = summ_tok.index_word.get(predicted_id, '')

        # Repetition Check: If used & long, skip to next best
        if word_candidate in seen_words and len(word_candidate) > 3:
            predicted_id = top_indices[1]

        word = summ_tok.index_word.get(predicted_id, '')

        if predicted_id == 0 or word == 'eostok':
            break

        if word and word != 'sostok':
            decoded_words.append(word)
            seen_words.add(word)

        if i < MAX_SUMMARY_LEN:
            decoder_input[0, i] = predicted_id

    return " ".join(decoded_words)

# ============== 2. ADVANCED Q&A LOGIC ==============
def answer_question(question, document_text):
    # Split sentences robustly
    sentences = re.split(r'(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|\?)\s', document_text)
```

```
        q_words = re.sub(r'[^\w\s]', '', question.lower()).split()

        # Stop Words (Common words to ignore for better matching)
        stop_words = {'the', 'is', 'at', 'which', 'on', 'in', 'a', 'an', 'and', 'or', 'of', 'to', 'was', 'were', 'case', 'court'

        keywords = [w for w in q_words if w not in stop_words]

        best_score = -1
        best_sentence = "Info not found in text."

        for sentence in sentences:
            if len(sentence.split()) < 4: continue

            s_lower = sentence.lower()
            score = 0

            for word in keywords:
                if word in s_lower:
                    score += 1
                    # Boost score if asking for numbers/facts and sentence has digits
                    if word in ['section', 'year', 'sentence', 'amount'] and any(c.isdigit() for c in sentence):
                        score += 2

            if score > best_score:
                best_score = score
                best_sentence = sentence.strip()

        return best_sentence

    # ============== MAIN EXECUTION ==============
    def main():
        model, text_tok, summ_tok = load_resources()
        time.sleep(1)
        clear_screen()
        print_header()

        # 1. Show Case
        print("\n[ INPUT LEGAL JUDGMENT ]")
        print("-" * 70)
        print(textwrap.fill(HARDCODED_CASE.strip(), width=70))
        print("-" * 70)

        # 2. Simulate Processing
        loading_bar("> Analyzing Text Features (Stacked BiLSTM Layer)...")

        # 3. Generate Summary
        summary = generate_summary(HARDCODED_CASE, model, text_tok, summ_tok)

        print("\n" + "="*70)
        print(" GENERATED SUMMARY:")
        print("=" * 70)
        formatted_summary = textwrap.fill(summary.capitalize(), width=70)
        type_effect(formatted_summary, delay=0.03)
        print("=" * 70)

        print("\n[ AUTO-Q&A SESSION STARTING ]")
        time.sleep(2)

        # 4. Auto Ask Questions
        for i, q in enumerate(HARDCODED_QUESTIONS, 1):
            print(f"\n[Question {i}]: {q}")
            time.sleep(1)

            answer = answer_question(q, HARDCODED_CASE)

            sys.stdout.write("Answer: ")
            type_effect(answer, delay=0.02)
            time.sleep(1.5)

        print("\n" + "="*70)
        print(" DEMONSTRATION COMPLETE")
        print("="*70)

        # Optional Manual Mode
        while True:
            user_in = input("\n(Optional) Type manual question or 'exit': ")
            if user_in.lower() == 'exit': break
            ans = answer_question(user_in, HARDCODED_CASE)
            print(f"Answer: {ans}")

    if __name__ == "__main__":
        main()
```

```
Loading AI Models... Please wait...
======================================================================
          LEGAL ASSISTANT AI (BiLSTM + ATTENTION)
======================================================================

[ INPUT LEGAL JUDGMENT ]
----------------------------------------------------------------------
The appellant herein was convicted by the Trial Court under Section
302 of the Indian Penal Code (IPC) for the murder of his wife, Sunita
Devi. The prosecution case is that on the night of 15th August 2018, a
quarrel broke out between the appellant and the deceased regarding a
financial dispute. The neighbors heard screams and alerted the local
police station.   Upon arrival, the police found the deceased lying in
a pool of blood with severe head injuries. The post-mortem report
confirmed that the cause of death was a heavy blow to the skull by a
blunt object, likely an iron rod recovered from the scene. The
appellant was arrested on the spot, and his clothes were found stained
with blood matching the blood group of the deceased.  The appellant
pleaded not guilty, claiming alibi, stating he was at his brother's
house during the incident. However, the prosecution produced three
independent witnesses, including a neighbor who saw the appellant
entering the house with an iron rod shortly before the screams were
heard. Furthermore, the forensic report confirmed the fingerprints on
the weapon matched the appellant.  The High Court dismissed the
appeal, upholding the life imprisonment sentence awarded by the
Sessions Court. The appellant has now approached this Court by special
leave. The learned counsel for the appellant argued that the evidence
was circumstantial and the extra-judicial confession was coerced.
However, considering the consistency in witness statements and
forensic evidence, the guilt of the accused is established beyond
reasonable doubt. The appeal is devoid of merit and is liable to be
dismissed.
----------------------------------------------------------------------

> Analyzing Text Features (Stacked BiLSTM Layer)...
[==============================] Done.

======================================================================
 GENERATED SUMMARY:
======================================================================
The supreme court of india delivered a judgment on the application of
section 302 ipc murder in a matter concerning writ petition civil the
application primarily addressed the question of fundamental rights the
ruling emphasized the principle of natural justice the final decision
was to appeal dismissed the lower affirmingreversing the matter courts
findings
======================================================================

[ AUTO-Q&A SESSION STARTING ]

[Question 1]: Who was convicted in this case?
Answer: The appellant herein was convicted by the Trial Court under Section 302 of the Indian Penal Code (IPC) for the murde

[Question 2]: What section of IPC was applied?
Answer: The appellant herein was convicted by the Trial Court under Section 302 of the Indian Penal Code (IPC) for the murde

[Question 3]: What was the cause of death?
Answer: The post-mortem report confirmed that the cause of death was a heavy blow to the skull by a blunt object, likely an

[Question 4]: Did the fingerprints match?
Answer: Furthermore, the forensic report confirmed the fingerprints on the weapon matched the appellant.

[Question 5]: What was the final verdict?
Answer: The appellant herein was convicted by the Trial Court under Section 302 of the Indian Penal Code (IPC) for the murde

======================================================================
 DEMONSTRATION COMPLETE
======================================================================
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipython-input-1081536575.py in <cell line: 0>()
    210
    211 if __name__ == "__main__":
--> 212     main()

 2 frames
/usr/local/lib/python3.12/dist-packages/ipykernel/kernelbase.py in _input_request(self, prompt, ident, parent, password)
   1217             except KeyboardInterrupt:
   1218                 # re-raise KeyboardInterrupt, to truncate traceback
-> 1219                 raise KeyboardInterrupt("Interrupted by user") from None
   1220             except Exception:
   1221                 self.log.warning("Invalid Message:", exc_info=True)

KeyboardInterrupt: Interrupted by user
```

```
Start coding or generate with AI.
```